



DOCKER & DOCKER COMPOSE

Microservices Containerization

ASSIGNMENT REPORT

Containerizing four Node.js microservices — User, Product, Order and Gateway — with individual Dockerfiles and orchestrating them through a single Docker Compose file on a shared network.

SUBMITTED BY	Tabrez Ajaz
OBJECTIVE	Containerize a microservices app using Node.js, Docker & Docker Compose
TOTAL MARKS	50
SERVICES	user (3000), product (3001), order (3002), gateway (3003)
REPOSITORY	github.com/TabrezAjaz/Microservices-Task

Table of Contents

1. Objective & Overview	
2. Application Architecture	
3. Step 1 — Dockerfile Creation	DONE
4. Step 2 — Docker Compose Configuration	DONE
5. Step 3 — Local Testing & Validation	DONE
6. Step 4 — Documentation & Screenshots	
7. Troubleshooting	
8. Conclusion	

1. Objective & Overview

What is being built and why.

1.1 Objective

Containerize a microservices-based application using **Node.js**, **Docker** and **Docker Compose**. Each of the four services gets its own Dockerfile, and all of them are started together with a single `docker compose up` command on a shared network so they can communicate.

1.2 Services

Service	Port	Endpoint	Purpose
<code>user-service</code>	3000	<code>/users</code>	Returns a list of users
<code>product-service</code>	3001	<code>/products</code>	Returns a list of products
<code>order-service</code>	3002	<code>/orders</code>	Lists / creates orders
<code>gateway-service</code>	3003	<code>/api/*</code>	API gateway that proxies the other three

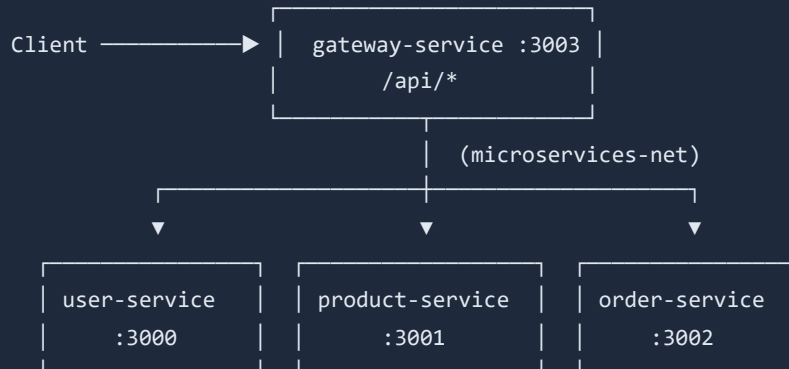
1.3 Deliverables

```
Microservices/  
├── user-service/      └── Dockerfile  
├── product-service/  └── Dockerfile  
├── order-service/    └── Dockerfile  
├── gateway-service/  └── Dockerfile  
├── docker-compose.yml  
└── README.md
```

2. Application Architecture

How the four containers talk to each other.

All four containers run on a single user-defined bridge network (`microservices-net`). The gateway calls the other services by their *service name* (Docker's built-in DNS), for example `http://user-service:3000` — no hard-coded IP addresses are required.



Client request	Gateway forwards to
GET /api/users	http://user-service:3000/users
GET /api/products	http://product-service:3001/products
GET /api/orders	http://order-service:3002/orders

3. Step 1 — Dockerfile Creation

15 marks

One Dockerfile per service (4 services × 5 marks).

Each Dockerfile does the following

- Uses an appropriate Node.js base image (`node:18-alpine` — small & secure)
- Sets a working directory (`/app`)
- Installs dependencies (`npm install`)
- Exposes the correct port
- Includes a valid `CMD` instruction (`node app.js`)

3.1 user-service/Dockerfile (port 3000)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

3.2 product-service/Dockerfile (port 3001)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3001
CMD ["node", "app.js"]
```

3.3 order-service/Dockerfile (port 3002)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3002
CMD ["node", "app.js"]
```

3.4 gateway-service/Dockerfile (port 3003)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3003
CMD ["node", "app.js"]
```

Why copy `package*.json` first? Docker caches each layer. Copying only the manifest before `npm install` means dependencies are re-installed only when they actually change — rebuilds after a code edit are much faster.

4. Step 2 — Docker Compose Configuration

10 marks

One file to define, network and run all four services.

The compose file

- Defines all four services
- Maps the correct host:container ports
- Configures a shared network (`microservices-net`) for communication

4.1 docker-compose.yml

```
services:
  user-service:
    build: ./user-service
    container_name: user-service
    ports:
      - "3000:3000"
    networks:
      - microservices-net
    restart: unless-stopped

  product-service:
    build: ./product-service
    container_name: product-service
    ports:
      - "3001:3001"
    networks:
      - microservices-net
    restart: unless-stopped

  order-service:
    build: ./order-service
    container_name: order-service
    ports:
      - "3002:3002"
    networks:
      - microservices-net
    restart: unless-stopped

  gateway-service:
    build: ./gateway-service
    container_name: gateway-service
    ports:
      - "3003:3003"
    depends_on:
      - user-service
      - product-service
      - order-service
    networks:
      - microservices-net
    restart: unless-stopped

networks:
  microservices-net:
    driver: bridge
```

depends_on makes the gateway start after the other three containers are created, and **restart: unless-stopped** keeps each service running across crashes and machine reboots.

5. Step 3 — Local Testing & Validation

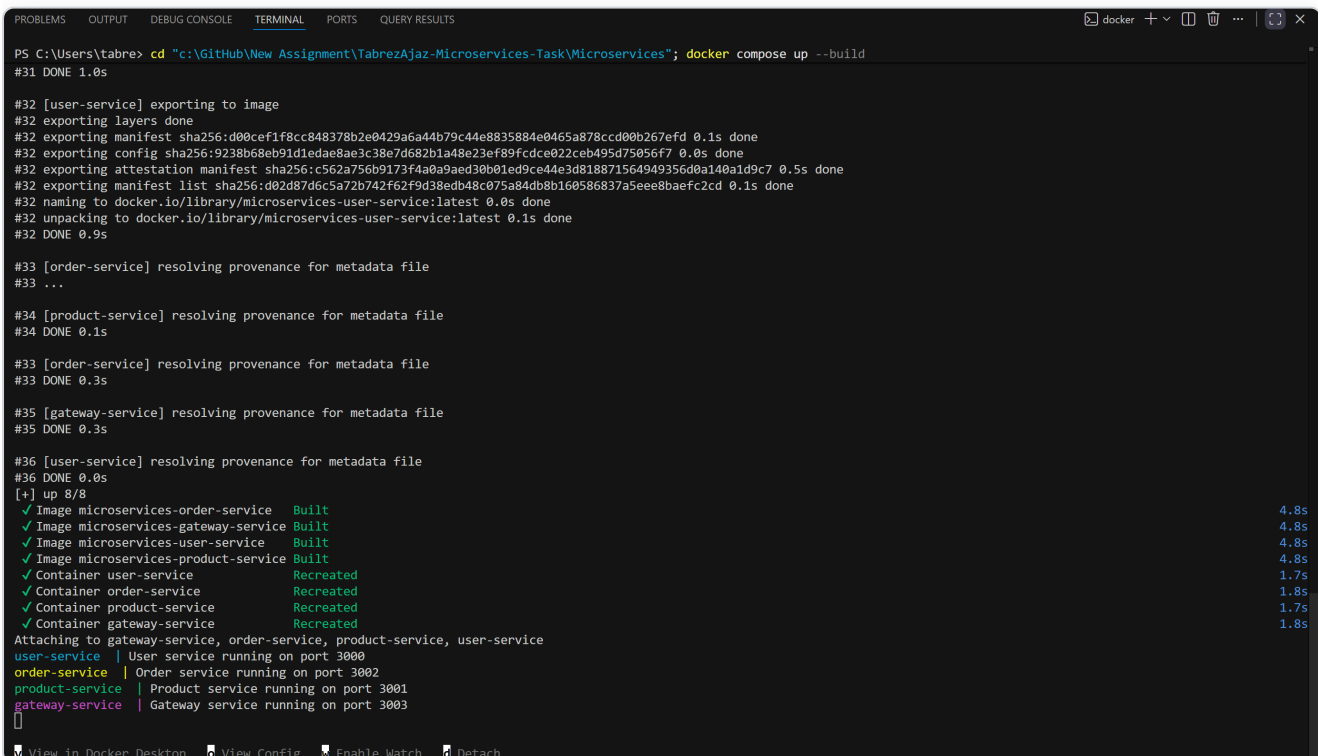
10 marks

Build, run, and confirm every service is accessible on localhost.

5.1 Build & start the stack

```
# From the Microservices/ folder
docker compose up --build

# Or run detached (in the background)
docker compose up --build -d
```



```
PS C:\Users\tabre> cd "c:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices"; docker compose up --build
#31 DONE 1.0s

#32 [user-service] exporting to image
#32 exporting layers done
#32 exporting manifest sha256:d00cef1f8cc848378b2e0429a6a44b79c44e8835884e0465a878ccd00b267efd 0.1s done
#32 exporting config sha256:9238b68eb91d1edae8ae3c38e7d682b1a48e23ef89fcdce022ceb495d75056f7 0.0s done
#32 exporting attestation manifest sha256:c562a756b9173f4a0a9aed30b01ed9ce44e3d818871564949356d0a140a1d9c7 0.5s done
#32 exporting manifest list sha256:d02d87d6c5a72b742f62f9d38edb48c075a84db8b160586837a5eee8baefc2cd 0.1s done
#32 naming to docker.io/library/microservices-user-service:latest 0.0s done
#32 unpacking to docker.io/library/microservices-user-service:latest 0.1s done
#32 DONE 0.9s

#33 [order-service] resolving provenance for metadata file
#33 ...

#34 [product-service] resolving provenance for metadata file
#34 DONE 0.1s

#33 [order-service] resolving provenance for metadata file
#33 DONE 0.3s

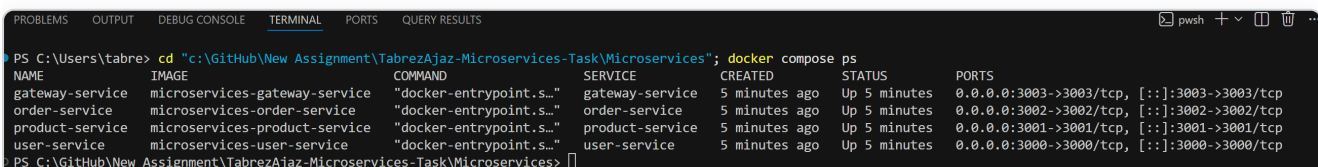
#35 [gateway-service] resolving provenance for metadata file
#35 DONE 0.3s

#36 [user-service] resolving provenance for metadata file
#36 DONE 0.0s
[+] up 8/8
✓ Image microservices-order-service Built 4.8s
✓ Image microservices-gateway-service Built 4.8s
✓ Image microservices-user-service Built 4.8s
✓ Image microservices-product-service Built 4.8s
✓ Container user-service Recreated 1.7s
✓ Container order-service Recreated 1.8s
✓ Container product-service Recreated 1.7s
✓ Container gateway-service Recreated 1.8s
Attaching to gateway-service, order-service, product-service, user-service
user-service | User service running on port 3000
order-service | Order service running on port 3002
product-service | Product service running on port 3001
gateway-service | Gateway service running on port 3003
[]
View in Docker Desktop View Config Enable Watch Detach
```

Figure 5.1 — `docker compose up --build` building and starting all four services.

5.2 Confirm containers are running

```
docker compose ps
```

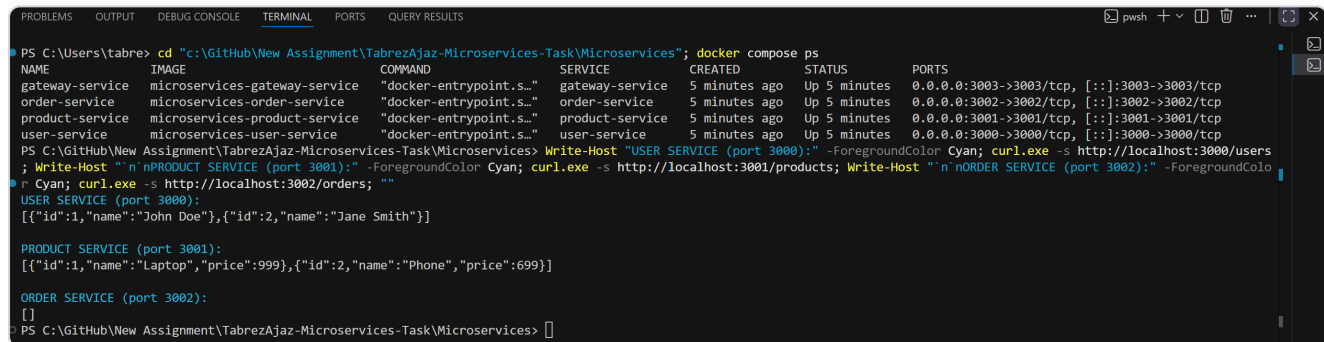


```
PS C:\Users\tabre> cd "c:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices"; docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
gateway-service     microservices-gateway-service      "docker-entrypoint.s..." gateway-service  5 minutes ago  Up 5 minutes  0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
order-service       microservices-order-service        "docker-entrypoint.s..." order-service   5 minutes ago  Up 5 minutes  0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
product-service     microservices-product-service      "docker-entrypoint.s..." product-service 5 minutes ago  Up 5 minutes  0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
user-service        microservices-user-service         "docker-entrypoint.s..." user-service    5 minutes ago  Up 5 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices>
```

Figure 5.2 — All four containers running with ports 3000–3003 mapped.

5.3 Test the direct service endpoints

```
curl http://localhost:3000/users      # [{"id":1,"name":"John Doe"}, ...]
curl http://localhost:3001/products  # [{"id":1,"name":"Laptop","price":999}, ...]
curl http://localhost:3002/orders    # []
```



```
PS C:\Users\tabre> cd "c:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices"; docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
gateway-service     microservices-gateway-service       "docker-entrypoint.s..." gateway-service  5 minutes ago Up 5 minutes 0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
order-service       microservices-order-service         "docker-entrypoint.s..." order-service   5 minutes ago Up 5 minutes 0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
product-service     microservices-product-service       "docker-entrypoint.s..." product-service 5 minutes ago Up 5 minutes 0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
user-service        microservices-user-service          "docker-entrypoint.s..." user-service    5 minutes ago Up 5 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices> Write-Host "USER SERVICE (port 3000):" -ForegroundColor Cyan; curl.exe -s http://localhost:3000/users
r Cyan; curl.exe -s http://localhost:3001/products; Write-Host "n nORDER SERVICE (port 3002):" -ForegroundColor
USER SERVICE (port 3000):
[{"id":1,"name":"John Doe"}, {"id":2,"name":"Jane Smith"}]

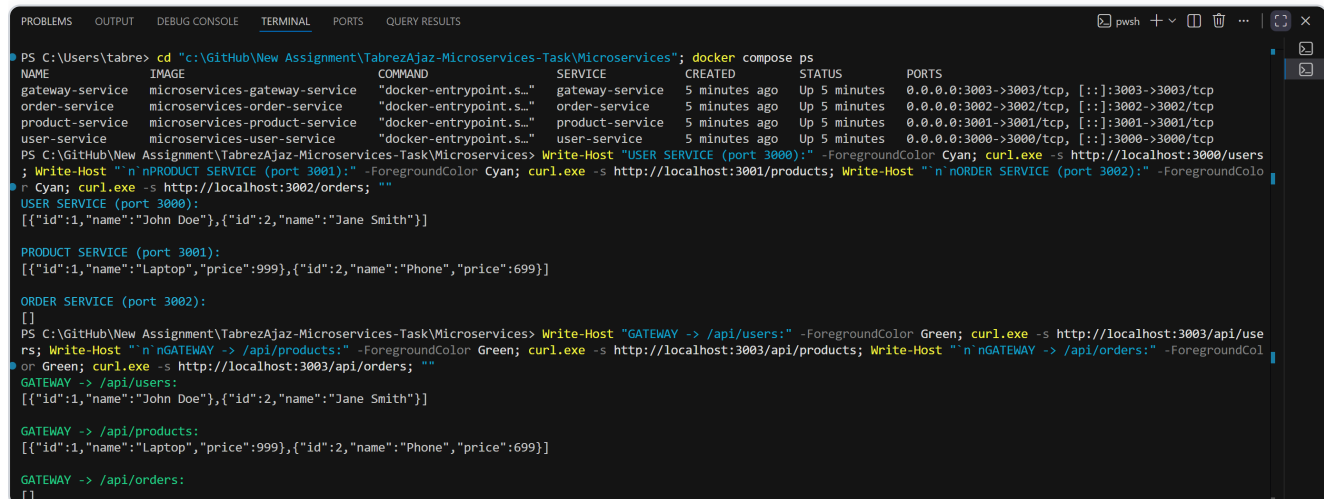
PRODUCT SERVICE (port 3001):
[{"id":1,"name":"Laptop","price":999}, {"id":2,"name":"Phone","price":699}]

ORDER SERVICE (port 3002):
[]
PS C:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices>
```

Figure 5.3 — User, Product and Order services responding on ports 3000–3002.

5.4 Test through the Gateway (/api)

```
curl http://localhost:3003/api/users
curl http://localhost:3003/api/products
curl http://localhost:3003/api/orders
```



```
PS C:\Users\tabre> cd "c:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices"; docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
gateway-service     microservices-gateway-service       "docker-entrypoint.s..." gateway-service  5 minutes ago Up 5 minutes 0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
order-service       microservices-order-service         "docker-entrypoint.s..." order-service   5 minutes ago Up 5 minutes 0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
product-service     microservices-product-service       "docker-entrypoint.s..." product-service 5 minutes ago Up 5 minutes 0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
user-service        microservices-user-service          "docker-entrypoint.s..." user-service    5 minutes ago Up 5 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices> Write-Host "USER SERVICE (port 3000):" -ForegroundColor Cyan; curl.exe -s http://localhost:3000/users
r Cyan; curl.exe -s http://localhost:3002/orders; Write-Host "n nPRODUCT SERVICE (port 3001):" -ForegroundColor
USER SERVICE (port 3000):
[{"id":1,"name":"John Doe"}, {"id":2,"name":"Jane Smith"}]

PRODUCT SERVICE (port 3001):
[{"id":1,"name":"Laptop","price":999}, {"id":2,"name":"Phone","price":699}]

ORDER SERVICE (port 3002):
[]
PS C:\GitHub\New Assignment\TabrezAjaz-Microservices-Task\Microservices> Write-Host "GATEWAY -> /api/users:" -ForegroundColor Green; curl.exe -s http://localhost:3003/api/use
rs; Write-Host "n nGATEWAY -> /api/products:" -ForegroundColor Green; curl.exe -s http://localhost:3003/api/products; Write-Host "n nGATEWAY -> /api/orders:" -ForegroundColor
or Green; curl.exe -s http://localhost:3003/api/orders; Write-Host "n nGATEWAY -> /api/users:" -ForegroundColor
GATEWAY -> /api/users:
[{"id":1,"name":"John Doe"}, {"id":2,"name":"Jane Smith"}]

GATEWAY -> /api/products:
[{"id":1,"name":"Laptop","price":999}, {"id":2,"name":"Phone","price":699}]

GATEWAY -> /api/orders:
[]
```

Figure 5.4 — Gateway service proxying all three backend services through /api/*.

Result: All four services build successfully, start together, and are reachable on **localhost** on ports 3000, 3001, 3002 and 3003 — both directly and through the gateway.

6. Step 4 — Documentation & Screenshots 10 marks

README with setup, testing and troubleshooting.

A `README.md` is included alongside the compose file. It documents the setup instructions, how to test each service, basic troubleshooting tips, and screenshots of the services running in Docker. Useful housekeeping commands:

```
docker compose logs -f          # tail logs from all services
docker compose logs gateway-service  # logs from one service
docker compose down             # stop and remove the containers
```

7. Troubleshooting

Common issues and quick fixes.

Problem	Cause / Fix
<code>port is already allocated</code>	Another process uses 3000–3003. Stop it, or change the host port in <code>docker-compose.yml</code> (e.g. <code>"3100:3000"</code>).
Gateway returns <code>{"error":"Error fetching users"}</code>	A downstream service didn't start. Check <code>docker compose logs user-service</code> (and product/order).
<code>Cannot connect to the Docker daemon</code>	Docker Desktop is not running. Start it and retry.
Code changes not reflected	Rebuild the images: <code>docker compose up --build</code> .
<code>docker compose</code> not found	Use the legacy hyphenated command <code>docker-compose up --build</code> .
Container exits immediately	Inspect logs: <code>docker compose logs <service></code> .

8. Conclusion

Summary of what was delivered.

All four Node.js microservices were containerized with individual Dockerfiles using the lightweight `node:18-alpine` base image, and orchestrated with a single `docker-compose.yml` on a shared `microservices-net` bridge network. Running `docker compose up --build` starts the entire stack, and every service is accessible on `localhost` — directly on ports 3000–3003 and through the gateway's `/api/*` routes. The accompanying `README.md` documents setup, testing and troubleshooting, completing all four assignment requirements.